Here is the implementation for **Stage 2: public route and unverified protected route** for both Express and FastAPI.

Node.js (Express & JavaScript)

javascript

```javascript
// Add these routes to your existing Express app from Stage 1

// GET /public/info
app.get('/public/info', (req, res) => {
  return res.status(200).json({ message: "Welcome stranger! This info is public." });
});

// GET /protected/profile
app.get('/protected/profile', (req, res) => {
  const authHeader = req.headers.authorization;

  // Validate the presence and structure of the Bearer token
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: "Access token required" });
  }

  const token = authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ error: "Access token required" });
  }

  // Token is present (unverified for now)
  return res.status(200).json({
    message: "You presented a token! Verification logic goes here in the next stage."
  });
});
```

Use code with caution.

Python (FastAPI)

python

```python
from fastapi import Request
# Add these routes to your existing FastAPI app from Stage 1

@app.get("/public/info", status_code=status.HTTP_200_OK)
def get_public_info():
    return {"message": "Welcome stranger! This info is public."}

@app.get("/protected/profile", status_code=status.HTTP_200_OK)
def get_protected_profile(request: Request):
    auth_header = request.headers.get("Authorization")

    # Validate the presence and structure of the Bearer token
    if not auth_header or not auth_header.startsWith("Bearer "):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Access token required"  # Match JSON output format requirement
        )

    try:
        token = auth_header.split(" ")[1]
        if not token:
            raise ValueError()
    except Exception:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Access token required"
        )

    # Token is present (unverified for now)
    return {"message": "You presented a token! Verification logic goes here in the next st
```

Use code with caution.

Note: For FastAPI, if you want your custom exception to output `{"error": "Access token required"}` instead of `{"detail": "..."}` to match the exact requirement, you can catch `HTTPException` in an exception handler or explicitly return a `JSONResponse(status_code=401, content={"error": "Access token required"})`.

Are you ready to move on to the next phase? Tell me what you'd like to do:

- Implement **Supabase token verification** (`supabase.auth.getUser(token)`) to secure the protected route
- Extract the route logic into reusable **middleware**
- Set up **CORS policies** to allow front-end connection